



Name:- soham kulkarni

Class:- Sy-A

RollNo.:-A-58

Batch:-A4

Aim:- To Perform different computations using NumPy library.

Assignment 4:-

1. Explain the Role of NumPy in the Python Scientific Computing Ecosystem.

Ans: NumPy plays a fundamental role in the Python scientific computing ecosystem by providing powerful tools for numerical and mathematical operations. It offers an efficient array structure that allows users to store and manipulate large amounts of numerical data. NumPy includes many built-in mathematical and statistical functions that make complex calculations easier and faster. It also serves as the core foundation for many other scientific and data analysis libraries in Python, enabling researchers, engineers, and data scientists to perform advanced computations efficiently.

2. Why is NumPy Considered Faster than Pure Python Computations?

Ans: NumPy is considered faster than pure Python computations because it uses optimized C-based implementations internally, which allow operations to be executed at a lower level in memory. While Python lists process elements one by one, NumPy arrays perform operations on entire arrays simultaneously through vectorized computations. This reduces the need for loops and improves execution speed. Additionally, NumPy arrays store data in contiguous memory blocks, which increases efficiency when performing numerical operations on large datasets.

3. What Types of Mathematical Operations Can Be Performed Using NumPy?

****Ans: **** NumPy supports a wide range of mathematical operations that are useful for numerical computing and data analysis. These operations include basic arithmetic operations such as addition, subtraction, multiplication, and division. It also supports advanced mathematical functions like square root, logarithmic functions, exponentiation, trigonometric functions, and matrix operations. In

addition, NumPy provides statistical operations such as mean, median, variance, standard deviation, and sum. These functions allow users to perform complex numerical calculations quickly and efficiently.

4. Explain Indexing and Slicing in NumPy Arrays.

Ans: Indexing and slicing in NumPy arrays are techniques used to access specific elements or subsets of data within an array. Indexing allows users to retrieve a single element from an array using its position or index value. In NumPy, indexing starts from zero, meaning the first element is accessed using index 0. Slicing allows users to extract a range of elements from an array by specifying a start index and an end index. It helps in selecting a portion of the array without modifying the original data. These features make it easier to manipulate and analyze specific parts of large datasets.

5. How Does NumPy Integrate with Other Libraries?

Ans: NumPy integrates closely with many other Python libraries used in data science and scientific computing.

a. With Pandas: NumPy arrays are used internally by Pandas to store and process numerical data efficiently. Many Pandas operations rely on NumPy functions for performing calculations and managing datasets.

b. With Matplotlib: NumPy arrays are commonly used as input data for creating graphs and visualizations in Matplotlib. Numerical results generated using NumPy can easily be plotted as charts, graphs, or histograms.

c. With Scikit-learn: In machine learning, Scikit-learn uses NumPy arrays to store datasets and perform mathematical operations required for training and evaluating models. Most machine learning algorithms in Scikit-learn accept NumPy arrays as input data.

```
In [ ]: import numpy as np
```

```
In [ ]: my_list = [10, 20, 30, 40, 50]

array = np.array(my_list)

print(array)

[10 20 30 40 50]
```

```
In [ ]: arr1 = np.array([10, 20, 30])
arr2 = np.array([1, 2, 3])
```

```
print("Addition:", arr1 + arr2)
print("Subtraction:", arr1 - arr2)
print("Multiplication:", arr1 * arr2)
print("Division:", arr1 / arr2)
```

```
Addition: [11 22 33]
Subtraction: [ 9 18 27]
Multiplication: [10 40 90]
Division: [10. 10. 10.]
```

```
In [ ]: arr = np.array([10, 20, 30, 40, 50])

print("Sum:", np.sum(arr))
print("Mean:", np.mean(arr))
```

```
Sum: 150
Mean: 30.0
```

```
In [ ]: arr = np.array([1, 2, 3, 4, 5, 6])

new_array = arr.reshape(2, 3)

print(new_array)
```

```
[[1 2 3]
 [4 5 6]]
```

```
In [ ]: arr = np.array([10, 20, 30, 40, 50])

print(arr[0])
print(arr[2])
print(arr[4])
```

```
10
30
50
```

```
In [ ]: A = np.array([[1, 2],
                      [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

# Matrix Addition
print("Addition:")
print(A + B)

# Matrix Multiplication
print("Multiplication:")
print(np.dot(A, B))
```

Addition:
[[6 8]
[10 12]]
Multiplication:
[[19 22]
[43 50]]

```
In [ ]: # Random numbers between 0 and 1
print(np.random.rand(5))

# Random integers
print(np.random.randint(1, 10, 5))
```

[0.12824075 0.30065683 0.31498563 0.99931117 0.27908469]
[5 2 2 9 8]

```
In [ ]: arr = np.array([10, 20, 30, 40, 50])

print("Variance:", np.var(arr))
print("Standard Deviation:", np.std(arr))
```

Variance: 200.0
Standard Deviation: 14.142135623730951

```
In [ ]: arr = np.array([1, 4, 9, 16])

print("Square Root:", np.sqrt(arr))
print("Square:", np.square(arr))
print("Log:", np.log(arr))
print("Exponential:", np.exp(arr))
```

Square Root: [1. 2. 3. 4.]
Square: [1 16 81 256]
Log: [0. 1.38629436 2.19722458 2.77258872]
Exponential: [2.71828183e+00 5.45981500e+01 8.10308393e+03 8.88611052e+06]

```
In [ ]: # Python list
list1 = [1, 2, 3, 4]

# NumPy array
array1 = np.array([1, 2, 3, 4])

print("Python List:", list1)
print("NumPy Array:", array1)
print("List * 2:", list1 * 2)
print("Array * 2:", array1 * 2)
```

Python List: [1, 2, 3, 4]
NumPy Array: [1 2 3 4]
List * 2: [1, 2, 3, 4, 1, 2, 3, 4]
Array * 2: [2 4 6 8]

Conclusion: This practical helped in understanding the basic numerical computations using the NumPy library. We learned how to create arrays, perform arithmetic operations, generate random numbers, and compute statistical values

such as mean, variance, and standard deviation. The practical also demonstrated the efficiency of NumPy arrays compared to Python lists for numerical calculations. Overall, it improved our understanding of performing fast and efficient mathematical operations in Python.